

AgentWorkShop: Governed Agent-Team Execution for Industrial Recipe Optimization

Anonymous Authors

Abstract—Industrial recipe optimization couples process objectives with equipment limits, heterogeneous protocols, and the authority to change operating conditions. Connecting an agent to a device does not, by itself, preserve these constraints or distinguish a successful command from a successful intervention. This paper presents AgentWorkShop, a supervisory execution framework that connects deployed agent-team members to engineering-level acquisition and control nodes through explicit capabilities and production context. A governed write path combines applicable operating limits and approval requirements, while intervention records link accepted writes to observations, submitted judgments, and separate compensation actions. Evaluation uses real protocol transports connected to simulated plants, with deterministic controllers and scripted teams to isolate execution behavior from language-model reasoning. The integrated archive passes 75 implementation checks, rejects all 24 configured out-of-constraint probes across four writable stacks, and accepts all four legal-write probes. A four-arm ablation attributes rejection of the tested recipe-window violations to the window check rather than readback verification. A recorded multi-node team mission reduces film thickness from 26.90 to 25.48 μm with four governed writes on three parameters, reaching a $25.0 \pm 0.7 \mu\text{m}$ objective. These results show how a common execution layer can preserve the distinction between action authority, device acceptance, and process outcome. The evidence supports simulation-tier supervisory integration; physical reliability and comparative agent optimization remain to be evaluated.

Index Terms—Agent teams, industrial informatics, recipe optimization, supervisory control, write governance.

I. INTRODUCTION

SUPERVISORY recipe adjustment links a production objective to changes on the line. Film thickness, for example, depends on screw speed, line speed, and temperatures; biaxial stretching couples adjustments across several sections. Choosing a setpoint is only one part of this task. Each intervention also needs execution authority, applicable operating limits, and observations that distinguish a completed command from an improved process.

Large language model (LLM) agents are being explored for industrial diagnosis, engineering assistance, and programmable logic controller (PLC) program generation [1]–[4], including interaction with simulated controllers and machine demonstrators [5], [6]. As agents coordinate interventions across equipment, protocol connectivity must be coupled to supervisory context: which deployed member may change which parameter, under which recipe, and with whose approval? Task orchestration and device access alone do not answer these questions or connect a write to its subsequent judgment.

AGENTWORKSHOP makes these relationships explicit through member–node bindings and shared context for lines,

products, recipes, and runs. Nodes expose engineering quantities rather than register addresses; bindings grant scoped tool authority; and intervention records associate changes with observations and submitted judgments. Fig. 1 contrasts this organization with application-specific integration. The aim is to make supervisory search inspectable across heterogeneous protocols while keeping fast regulation and plant protection outside the agent loop.

At seconds-to-minutes timescales, manual bindings require approval and automatic bindings retain execution checks without that wait. Separate judgment and compensation operations distinguish rejection, driver acceptance, readback agreement where supported, process response, and attempted restoration. These distinctions define three contributions:

- 1) A **node-native supervisory model** connecting heterogeneous acquisition and control nodes, deployed members, and production context through explicit capabilities and shared identifiers (Secs. III and IV-A).
- 2) An **implemented governed loop** linking source-dependent admission, intervention records with optional hypotheses, submitted judgments, separate compensation, and human approval (Sec. IV).
- 3) A **layered, traceable evaluation** using protocol checks, an interlock ablation, fixed-controller trials, and scripted team missions to characterize constraint enforcement and multi-node loop closure (Sec. V).

The evaluation uses real protocol transports with simulated plants and deterministic policies, isolating supervisory execution from LLM reasoning. The contribution is an execution and evidence framework, not a new control law or a claim of field superiority. Configured recipe windows constrain proposals; neither they nor shared context provide atomic execution or a physical safety guarantee.

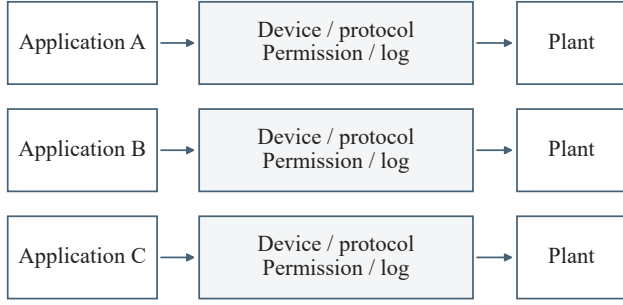
II. RELATED WORK

A. Industrial Agents and Closed-Loop Applications

Agent-based manufacturing control provides a foundation for connecting team roles to industrial capabilities [7]. Industrial LLM applications span knowledge-graph construction [8], diagnosis [9], PLC generation and verification [3], [4], and digital-twin configuration [10]. Natural-language machine control over OPC UA [6], simulated fault-tolerant control, and twin-supported fault handling [5], [11] already combine corrective actions with process knowledge, deterministic checks, or simulation. InstructMPC introduces language-derived context into model-predictive control [12]. Agent-based supervision of service-oriented industrial cyber-physical

(a) Application-specific integrations

Mappings replicated for each application

**(b) Explicit member-to-node binding**

Node-scoped access through a shared interface

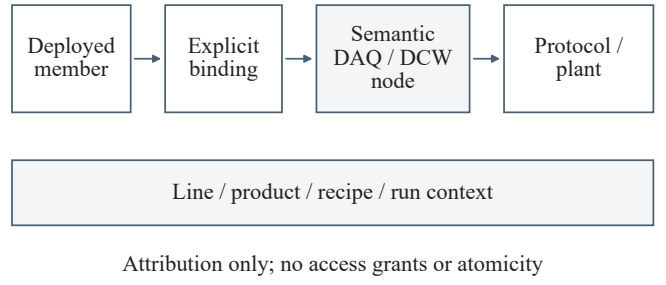


Fig. 1. Conceptual supervisory integration: (a) conventional application-specific connections; (b) explicit member–node capabilities and shared production context in the proposed organization. PLC regulation remains outside the agent loop. The panels illustrate design patterns, not an empirical comparison of competing systems.

systems [13] and LLM-based shopfloor coordination [14] are particularly close to our setting. Our focus is the connection between deployed-member authority, source-dependent execution checks, and production-context attribution, rather than execution checks or agent coordination alone.

B. Orchestration, Industrial Models, and Execution Governance

AutoGen, LangGraph, and CrewAI organize agent tasks and tools [15]–[17]; Model Context Protocol (MCP) and Agent-to-Agent (A2A) standardize tool and inter-agent interfaces [18], [19]. Industrial supervision adds node semantics, driver behavior, and operating context. Our organization of lines, products, recipes, and runs draws on ISA-95 work units, product definitions, and performance records, and the ISA-88 master-recipe concept [20], [21], without implementing their full procedural models. Semantic asset models and gateway digital twins abstract heterogeneous equipment [22]–[24]; we address write governance over that abstraction.

Permissions, range validation, approval, and audit trails have counterparts in DCS/SCADA/MES practice and IEC 62443-3-3 controls [25]. We connect these functions to deployed member instances and associate interventions with judgments across engine boundaries. Formal action shielding and control-barrier methods [26], [27], and simplex-style run-time assurance [28], instead target protected execution with specified guarantees. Our admission checks and rule-based backstop do not provide those guarantees. Human supervision also requires a division of responsibility beyond an approval dialog [29], [30].

C. Evaluation of Tools and Industrial Control

AgentBench and consequential tool-use benchmarks assess agent behavior and policy compliance [31], [32]; the Tennessee Eastman Process (TEP) and Secure Water Treatment (SWaT) support process-monitoring research [33], [34]. Our evaluation separates source checks, mock-driver probes, software-protocol integration, and scripted optimization, so

each result identifies the execution layer it tests. Indirect prompt injection [35] and benchmarked defenses [36] motivate testing compromised agents’ use of legitimate tools; adversarial LLM probes remain outside this evaluation.

III. SYSTEM ARCHITECTURE

AGENTWORKSHOP separates team coordination from plant interaction while linking both through industrial nodes and production context (Fig. 2). A node is an acquisition or control endpoint with engineering semantics, connection settings, line membership, and agent bindings. Channels coordinate agents; controller services mediate node operations. This separates three responsibilities: deciding what to change, authorizing and executing the change, and recording what followed.

A. A Shared Supervisory Core

The Node.js core hosts team routing, industrial services, and protocol adapters. Operator and integration interfaces share objects but differ in permissions; agent-tool authority comes from node bindings, not channel access or endpoint availability. Engine subprocesses and persistent stores remain external to the core. Connections, protocol queues, and readback are asynchronous; fast regulation and protection remain with the plant controller and its safety equipment.

B. Reasoning, Plant Interaction, and Operational Context

A channel coordinates deployed agent instances, called members, through a lead, workers, a task tree, messages, and retrievable memory. An agent proposes actions; its engine is the runtime implementation that executes it. Engine adapters preserve node identity and the controller interface across runtime implementations; the benchmark harness scripts these interactions for evaluation.

Acquisition nodes (DAQ) convert protocol readings to engineering quantities; control nodes (data-control-write, DCW) translate setpoints into device commands and readback results. Per-node runtimes manage timing and in-flight operations; controller services supply gateway state, recipe context, and

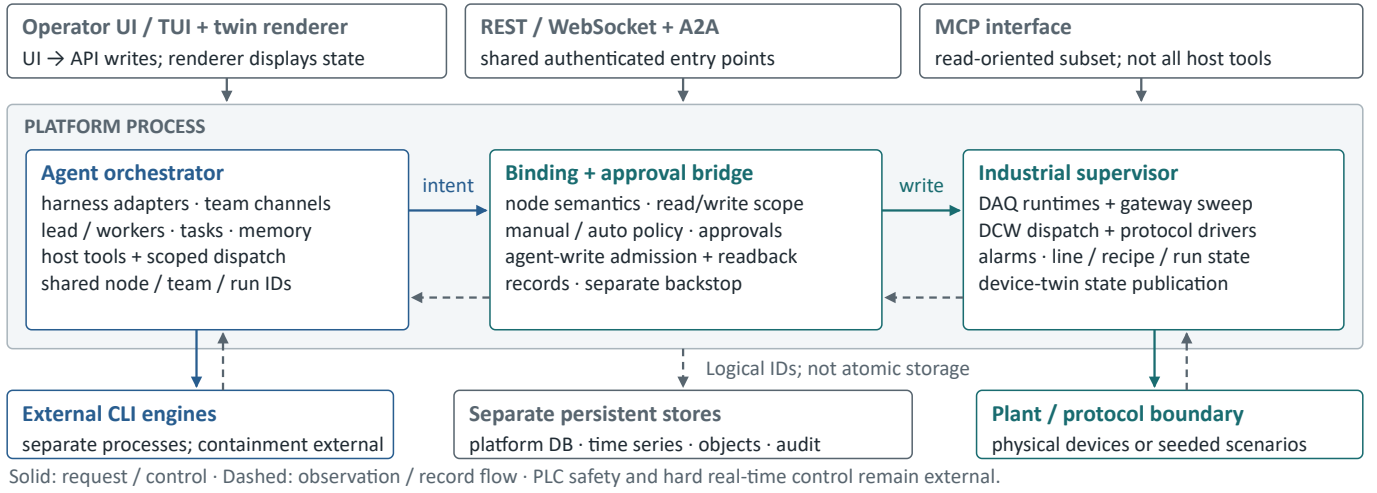


Fig. 2. System architecture and execution boundaries. The co-resident platform process hosts the agent orchestrator, the binding-and-approval bridge, and the industrial supervisor; operator entry points sit above it, and external CLI engines, persistent stores, and the plant boundary sit outside it. Shared identifiers support attribution across the logical data model; they do not make storage or physical actuation transactional.

write bookkeeping. Many-to-many node–device associations describe configuration, not execution routing: each control write uses the node’s configured driver and connection.

C. Logical Continuity Across a Production Run

Lines, products, recipes, and runs organize production context. Acquired data carry active-context tags resolved at ingestion. Optimization records retain line, node, recipe, and actor references, but no run identifier; their baseline and response queries use node, line, and time rather than enforce run-isolated observations, so a baseline can overlap an earlier run. Platform state uses SQLite and file-backed repositories; configured measurement and object backends are TimescaleDB and MinIO, with SQLite and disk fallbacks. These links support attribution across stores, not a single transaction domain.

A BOPET thickness adjustment illustrates how the objects work together. The channel carries one production objective and delegates a task to a worker; the worker queries a thickness-acquisition node, then addresses casting speed, fast-roll speed, or rail width through bound control nodes. Each node resolves its own protocol and engineering units, while the production objects identify the operating context. An optimization record associates an executed change with its actor and later judgment, so the task can span protocols without treating a channel message as write authority or a task-board completion as a device acknowledgment; Sec. V-E follows this sequence in the recorded mission.

IV. CORE MECHANISMS

A supervisory intervention passes through three decisions: whether the request may execute, what the device reports, and how the resulting process response should be judged. The platform represents these decisions separately so that neither a completed team task nor an accepted command stands in for process success. Fig. 3 summarizes this sequence; the

operator console exposes its commissioning, approval, and audit objects.

A. Optimization Objective and Search Space

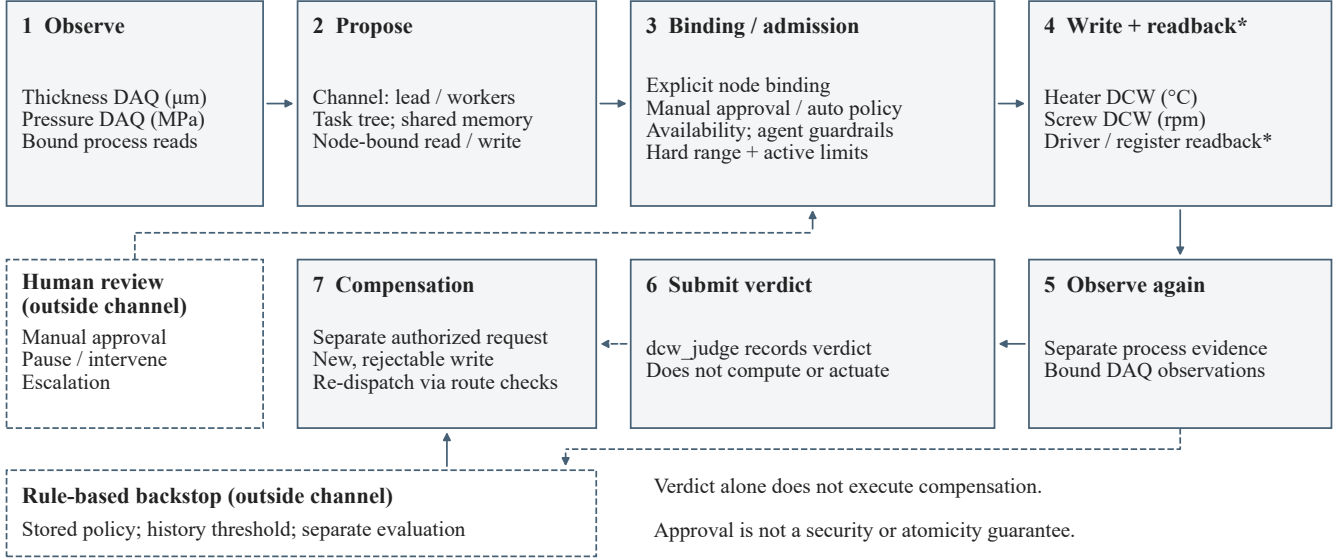
The supervisory task is to adjust recipe parameters toward a product objective under configured operating constraints. The strategy chooses the next proposal and its stopping rule; the platform governs execution and retains the evidence needed to inspect each iteration.

Decision variables. The decision variables are recipe parameters bound to a line’s control nodes, such as screw speed, line speed, and zone temperatures for cast film. A parameter is written in engineering units and resolved to a register only inside the controller (Sec. IV-C), so a proposal carries physical meaning rather than an address. A commissioned node defines the quantity, units, calibration, limits, and line membership; protocol differences remain inside its driver. Member–node bindings, not channel access or protocol connectivity, grant agent-tool authority; they name deployed member instances rather than templates. Newly instantiated members do not inherit existing bindings; the bindings themselves remain administratively mutable.

Objective and feasible set. A strategy defines its product objective and process constraints using acquisition data and engineering quantities. The team mission targets a film thickness band; the fixed-controller experiment combines thickness, defects, and setpoint-based energy and throughput proxies, subject to temperature and pressure constraints (Sec. V-D). Process effects require observations rather than write acknowledgments; freshness is exposed, but observation-quality gating is not enforced. The admissible setpoint interval is the intersection of the node’s hard range, active recipe window, and applicable product or baseline limits (Sec. IV-C). This interval constrains proposals; satisfying it neither certifies process feasibility nor discovers a new recipe window.

Iteration and termination. A hypothesis record links a proposal and write to an observation window and a submitted

Ordinary agent setpoint path; external intervention remains separate



* Readback only where supported; setpoint agreement is not process success.

Active limits include recipe windows where applicable; recipe / heartbeat checks are source-specific.

Fig. 3. The governed supervisory loop for ordinary agent setpoint requests. The channel coordinates a lead and workers whose per-node bindings gate admission—bound scope, availability, approval policy, hard range plus the active recipe window—before a driver write with register readback where the protocol supports it. `dcw_judge` records the agent’s verdict but neither computes it nor actuates; compensation is a separate, rejectable write; the human authority and the auto backstop sit outside the channel. Numbered items are loop steps. The evaluated plant is simulated; recipe dispatch and hold-heartbeat routes have the distinct admission behavior described in Sec. IV-C.

judgment. The supervisory strategy may stop at its acceptance band, on a favorable judgment, or at its write budget; these are policy decisions, not proofs of convergence.

B. The Agent Channel and Operator Surfaces

Channels route tasks among deployed members and retain messages and rationale. Task completion neither grants authority nor proves device execution. Industrial tools resolve the invoking member’s bindings for writes, judgments, and rollback; channel access and retrieved memory are not authorization. The scripted `agent-param-tools` check tests binding denial on the parameter layer (Sec. V-B).

C. Node Control Execution and the Parameter Layer

The parameter layer resolves semantic keys and units to control nodes and optional baseline limits, with active-product and recipe limits resolved dynamically, so agent operations need no register address, data type, or driver configuration. The semantic card supplies the engineering context—quantity, unit, precision, range, current value, freshness, and available recipe information—and its suggested step size is advisory, not an admission rule. Acquisition runtimes track due times and in-flight samples under a rotating gateway sweep, providing supervisory rather than hard real-time sampling.

For an ordinary agent setpoint request, the allowed value lies in

$$\mathcal{I}_n(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \quad (1)$$

where $\mathcal{H}_n$ is the node engineering range and the other intervals are applicable parameter-baseline, active-product, and active-recipe limits. An absent optional limit adds no restriction. This notation describes the implemented interval checks, not a model of plant feasibility. Entry routes differ: recipe dispatch supplies a run identifier and omits the recipe-window layer, while hold heartbeats bypass controller admission but share runtime write exclusion. Agent-origin ownership and directional cooldown checks are not universal guards. Line start installs an active run before dispatching recipe parameters individually. Stop clears that context, prevents subsequent acquisition dispatch, closes up to 500 open optimization records, and requests acquisition-offline status; it does not cancel in-flight samples, stop an already-running recipe dispatch, or block all control writes. Run tags are resolved at ingestion, so this command is supervisory batch management, not a PLC emergency stop.

Calibration maps engineering setpoints into the driver domain and available readback into physical units. Modbus and OPC UA drivers check readback using

$$\tau_n = \max(0.5 \cdot 10^{-d_n}, 0.005(u_n - l_n)), \quad |v_{rb} - v| \leq \tau_n, \quad (2)$$

where $[l_n, u_n]$ is the node range and d_n its display precision. For a nonzero linear calibration scale a , the driver receives $\tau_n/|a|$. Driver success is protocol-dependent: HTTP accepts a successful response without tolerance-checking an optional returned value, whereas MQTT publication supplies no readback. Only a validated readback supports setpoint agreement; neither that agreement nor request acceptance establishes pro-

cess success. A failed readback can follow an already executed write; it does not establish that the device was left unchanged.

Agent setpoint tools resolve bindings and obtain any required approval before controller admission, rechecking binding existence after approval. Admission checks pause and disable state, applicable ownership and directional cooldown, and the limits above; the runtime then handles write exclusion, connection, write, and readback. REST, recipe, and recovery routes have distinct entry authorization. Asynchronous connection and protocol queues prevent check-act atomicity against recipe changes, and write bookkeeping can fail without reversing device execution, so attribution is not an atomic commitment across storage and the device.

D. Agent Closed-Loop Control

An intervention connects observation, proposal, execution, judgment, and possible compensation. The agent may attach an expected effect to its request, but the hypothesis field is optional. After driver-reported success, bookkeeping attempts to retain a change anchor. Agent- and rollback-origin writes can additionally open an optimization record with asynchronous baseline statistics; manual and recipe writes do not. The evidence associates a change with later observations; it is neither a causal experiment nor a guarantee that every accepted write was captured.

Recording and coordination. The five-second same-target rule suppresses bookkeeping, not the physical command: the driver has already executed when the latest node anchor is checked. Otherwise, the node's previous open record is closed before unchanged-value suppression; an unchanged command can therefore close a record without replacing it. Ownership is node-scoped: an agent-origin write can be blocked by another agent's non-stale open record. Directional cooldown restricts immediate movement away from a recently restored value; it does not exclude all manual, recipe, or system interventions. Defaults are a 600-s baseline lookback, a 300-s cooldown, and a 30-min stale-record interval. A missing prior value or a bookkeeping failure can leave recovery or attribution incomplete even when the driver accepted the request.

Judgment. The judgment tool records the agent's reading of that comparison; it does not establish process success. An invocation of `dcw_judge` supplies a record identifier, verdict, and reason. A *keep* closes the record; for non-takeover judgments it also attempts to mark the currently active run as good, which is bookkeeping status rather than a certified snapshot of the operating point. *Rollback* and *uncertain* leave the record open. Setpoint restoration requires a separate rollback operation, and the strategy, not this status alone, decides whether to stop.

Rule-based backstop. While the control gateway is running, its sweep schedules asynchronous checks of open records whose stored policy is not observe-only, by default no earlier than 120 s after the record is set and no more often than every 30 s. The manager prefers acquisition streams sharing the control node's primary-device alias, falling back to the line. It queries from the record's set time to now and sums out-of-window returned points or bucket averages across channels;

a total of at least $b = 3$ triggers a rollback judgment. These need not be consecutive points, and b counts neither evaluator invocations nor raw sensor samples. Automatic recovery is attempted only while fewer than $K = 2$ retained records for the node have status `rolled-back`; the count spans retained history, not the current run, and does not bound every rollback route. At the threshold, or under an approval-required policy, the backstop records escalation rather than writing. Query aggregation, asynchronous execution, and later record-state changes prevent these rules from establishing a worst-case recovery deadline or cancelling already-pending recovery.

E. Recipe Management and Physical Rollback

Recipe history preserves intended operation, not achieved device state. Versioned edits and known-good snapshots retain parameter sets and provenance, but reverting a definition creates a version with restored metadata without restoring PLC values. Applying a recipe creates a run and dispatches parameters; starting a line also installs the active context, so definition, run metadata, and physical parameters need not agree transactionally. Physical rollback issues a new, rejectable write to a recorded baseline or anchor-derived target; the run-related route uses the previous value of the last retained pre-start changed anchor, which need not be the actual setpoint at run start. The rollback operation closes an originally open record before dispatch, marks it `rolled-back` on success, and may open a new record for the compensating write. A failed restoration does not reopen the original for sweep-based retry. Across nodes, compensation can partially succeed and cannot erase intermediate process states; restoring metadata, restoring a setpoint, and recovering the process are distinct outcomes.

F. Human-in-the-Loop Control

Manual bindings add a timed approval wait to agent setpoint and physical rollback requests; auto bindings omit the wait but retain applicable execution checks. Reads, verdict recording, and bound-node recipe-window editing do not require this physical-write approval. Setpoint requests show the member, node, requested value, request-time effective interval, and remaining time; rollback requests describe the target record or anchor. Approval reserves neither limits nor device state: binding existence and controller admission are checked again before execution. Expiry prevents execution and is recorded separately from refusal; duplicate pending requests for the same agent and node are suppressed, and unbinding can cancel outstanding requests. The decision endpoint authenticates a user but does not enforce a line- or owner-scoped approver role. Existing optimization records retain their creation-time recovery policy; changing a binding to manual does not retroactively change that policy. Tool approval differs from an optimization record awaiting human action: escalation does not execute recovery, and neither path covers an absent operator when $K = 2$ is reached. External engines still need deployment-level privilege and network restrictions: bindings govern sanctioned tool calls, not arbitrary I/O by a compromised process.

V. EXPERIMENTAL EVALUATION

A. Research Questions and Experimental Design

The evaluation addresses three questions: **Q1**, are configured write constraints enforced across heterogeneous protocols? **Q2**, what distinct roles do admission, readback, and process observations play? **Q3**, can scoped interventions close a team-level objective while retaining attributable execution records? These questions assess the execution framework, not the quality of an agent’s reasoning policy.

The primary integrated run is archive *B* (20260920094610-t04), retained with the September 20, 2026 report. It records seed 42, platform commit 1585948, and fingerprint b203077a581aaba1 over nine benchmark-harness files. The runtime is Node v24.19.0 on Windows x64. The invocation is `node bench/pipeline.mjs` with `--profile` integrated, `--seed` 42, and `--cl-seeds` 3. The immediately preceding same-version archive *A* is reported in Sec. V-F, including its different endpoint; neither run is selected for a favorable outcome. Ablation and protocol/fault results use separate archives and are not pooled with the integrated checks. The controller averages up to six latest returned history-point means. The BOPET mission averages up to five for thickness and four for melt temperature; these REST queries request one-second buckets. The first mission instead uses setpoint-register readback for its reported endpoint. The archive preserves the resulting aggregates, not raw buckets or their timestamps.

The testbed runs actual Modbus TCP, OPC UA, MQTT, HTTP, and Modbus RTU-over-TCP transports against simulated plants, including a cast-film physics model, a strategy-based film line, and a biaxially oriented polyethylene terephthalate (BOPET) line. The integrated tool harness is deterministic `opcode`; mission leads and workers are scripted mock-engine members, and LLM decisions are disabled. Thus, the experiments evaluate supervisory execution and its evidence trail, not learned optimization, physical production performance, or an advantage over another agent framework. The simulator-derived reference and author-defined probes are internal oracles, not independent industrial benchmarks.

Fig. 3 locates the measured stages within one supervisory iteration. We first test protocol coverage and admission (Q1), then distinguish window prevention from readback and process response (Q2), and finally examine fixed-controller and team-level loop closure (Q3). Each layer has its own unit of evidence: implementation checks, configured violation probes, controller seeds, or an intervention sequence. Their counts are not combined into a statistical success rate.

B. Q1: Constraint Enforcement Across Protocols

Archive *B* reports 75 passes, zero warnings, and zero failures across 17 phases. A pass is a check verdict, not an independent trial or a statistical confidence statement. The checks partition by function as 9 plant and provisioning, 5 protocol I/O and write governance, 7 three-cycle tool loops, 12 records, approval, recipes, and parameters, 16 team objectives and the BOPET line, 22 controller, scenario reuse, and backstop, and 4 multimodal acquisition and services.

TABLE I

PROTOCOL MEASUREMENTS IN *B*. LATENCY QUANTILES USE SIX TIMED WRITES PER STACK: P50 IS THE FOURTH SORTED VALUE AND P95 THE MAXIMUM. READBACK IS ONE SAMPLED DEVIATION, NOT AN ACCURACY BOUND. A DASH MEANS UNMEASURED OR INAPPLICABLE.

Stack	History points	p50/p95 (ms)	Readback
Modbus TCP	9	49.232/101.164	0.04 °C
OPC UA	11	19.437/22.110	0 rpm
MQTT	12	19.427/23.084	—
HTTP	13	32.941/33.546	—
RTU over TCP	14	—	—

Acquisition-only nodes have no write endpoint; three omitted write/governance sub-checks must not be counted as tested writes. The optional LLM phase is absent. A separate source audit (20260920095700-1e3s) passes three checks, including 20/20 code-constant anchors, and the accompanying `_r7-apilive.log` records 60 API passes and two persistence-related skips despite the consolidated report’s zero-skip summary; these overlapping layers are not summed as independent experiments.

Queries to five acquisition nodes return 59 bucketed history points, not 59 independently counted raw samples. Four writable stacks each reject six out-of-constraint probes, giving 24/24 rejections and zero false blocks on four legal writes; the checker counts HTTP error responses to configured hard-range and window probes, so it does not verify each physical non-effect or establish adversarial robustness. Reprovisioning the second film-line scenario requires no driver-code changes: its three writable lines reject 9/9 probes without false blocks, alongside two acquisition-only satellites.

Table I retains the per-protocol measurements instead of reducing integration quality to one score. The report’s 30.259-ms aggregate is the arithmetic mean of the four protocol p50 values, not the median of pooled writes. MQTT and HTTP archive no measured register readback, so their transport acceptance must not be described as verified actuator state. The approval request-to-decision latency is 448 ms with a scripted approver; human response time is not measured. The backstop rollback verdict is first observed at 130.205 s, and baseline restoration is verified subsequently, after an additional 1.5-s wait and read; the reported interval is therefore verdict-observation latency, not measured restoration time or a hard recovery deadline.

C. Q2: Prevention, Readback, and Process Response

The four-arm ablation uses seed 42 and three repetitions per arm (archive 20260920094442-e11ite). These are benchmark-only switches requiring `AW_BENCH_MODE=1`, not normal operator controls: the ablation bypasses soft-limit enforcement, readback-error enforcement, or both while retaining the engineering-range guard, and only the recipe soft layer is populated in this fixture. The report label “No readback” therefore means the tolerance is relaxed, not that readback I/O stops, and because the mock driver returns the submitted value without checking tolerance the switch is inert for mismatch detection here. The experiment isolates window rejection, not

TABLE II

FOUR-ARM GOVERNANCE ABLATION (THREE REPETITIONS, 18 OUT-OF-CONSTRAINT PROBES PER ARM). “BOUNDARY” COUNTS DECISIONS MATCHING THE ARM-SPECIFIC BOUNDARY ORACLE; “BREACHES” COUNTS EXECUTED WINDOW VIOLATIONS AMONG THE ATTACK PROBES ONLY. BOUNDARY CASES ARE SEPARATE; THE ENGINEERING GUARD REMAINS ACTIVE IN EVERY ARM.

Arm	Rejected	Breaches	False blocks	Boundary
Full	18/18	0	0	9/9
No window check	12/18	6	0	9/9
Relaxed readback	18/18	0	0	9/9
Both changes	12/18	6	0	9/9

the effectiveness of readback verification. Table II shows all arms, including the executed violations: each repetition runs six out-of-constraint probes and evaluates three legal and three boundary cases separately, on new fixture objects from the same design. Arm-specific seeds vary numerical probes within the same case families, so these are not paired identical treatments or independent physical-plant trials. Table II names the switches by what they change; “relaxed readback” means relaxed error enforcement, not omitted I/O.

Removing the recipe-window check admits both window-class violations in each repetition, whereas relaxing readback-error enforcement does not change rejection (Table II). This supports a specific mechanistic claim: the window check prevents these violations; readback and journaling do not substitute for it. The fourth arm’s “Ungated” label is not guard-free: engineering limits remain enforced. All nine legal writes per arm are accepted. Boundary agreement uses an arm-specific oracle, not a common safety criterion: full and relaxed-readback arms reject six of nine boundary cases, whereas the two window-disabled arms accept all nine. Thus, each window-disabled arm also executes six out-of-window boundary cases in addition to its six attack-group breaches. Per-repetition p50 ranges are 110.6–141.0, 114.8–127.2, 111.9–127.1, and 124.8–139.9 ms in table order; each quantile uses 23 successful writes (three legal and 20 timed), and arms run in fixed order. These descriptive ranges use mock in-memory readback, including in the verification-bypassed arms. The driver delay uses unseeded randomness, so the probe seed does not make these timings reproducible. They establish neither latency equivalence nor the cost of removing physical readback I/O.

A separate software-protocol drill tests why process observations are needed after a command is accepted. The archive 20260920094335-1b5g passes 13 checks (three source checks, six API checks, and four protocol checks); all five driver tests succeed. Its Modbus drill accepts an initial 182 °C write in 25 ms and reads back 182. Five rejected probes and a further legal write to 180 °C precede process polling. The first qualifying observation is 179.5 °C at a rounded 30 s from the start of polling, satisfying $|PV - 182| \leq 3$ °C. This sequence is a workflow check, not an isolated step-response measurement. After a frozen-process injection and subsequent write, the checker first observes a matching alarm at a rounded 2 s from its post-write timer, not 2 s from fault injection. A TCP disconnect/reconnect drill detects and recovers the

TABLE III

FIXED-CONTROLLER OBSERVATIONS IN B . J_1 AND J_2 FOLLOW THE FIRST AND SECOND WRITES; $\bar{J} = (J_1 + J_2)/2$ IS COMPUTED BEFORE ROUNDING. EACH SEED HAS TWO ACCEPTED CONTROLLER WRITES AND NO REJECTIONS; INITIALIZATION WRITES ARE EXCLUDED.

Seed	J_0	J_1	J_2	$\bar{J}/J^*$	Final h (μm)
42	70.167	87.288	86.402	0.966	49.200
43	68.608	87.446	87.027	0.970	49.683
44	65.740	87.447	86.713	0.969	50.100

connection. These narrowly exercised failures support the distinction between an accepted command and a responding process, not general fault tolerance.

D. Q3: Loop Closure With a Fixed Controller

To isolate the execution path from agent strategy, the cast-film controller starts at screw speed $N = 150$ rpm, line speed $v = 95$ m/min, and zone setpoint 200 °C. The reward is

$$J = 55J_{\text{thick}} + 25J_{\text{quality}} + 8J_{\text{energy}} + 7J_{\text{throughput}}, \quad (3)$$

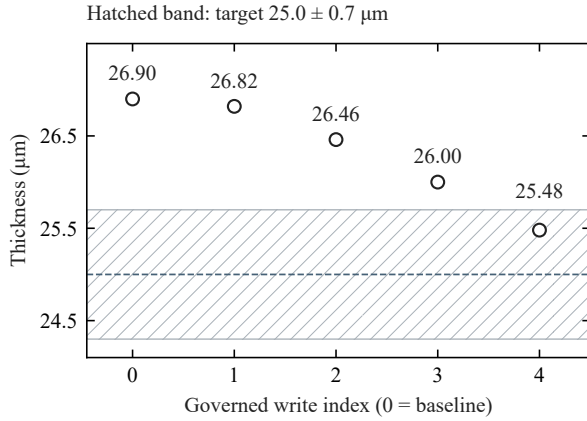
subject to melt temperature in $[195, 225]$ °C and pressure at most 22 MPa. For thickness h in μm, defect percentage d in percentage points, and commanded speeds N and v in rpm and m/min, respectively,

$$\begin{aligned} J_{\text{thick}} &= \max\left\{0, 1 - \frac{\max\{|h - 50| - 2, 0\}}{10}\right\}, \\ J_{\text{quality}} &= 1 - \frac{\min\{d, 8\}}{8}, \\ J_{\text{energy}} &= 1 - \frac{N - 50}{150}, \quad J_{\text{throughput}} = \frac{v}{120}. \end{aligned} \quad (4)$$

On the configured nonnegative-defect and command domain, $J \in [0, 95]$; infeasible states are unscored. The denominator is the stored, three-decimal reference score, not a rescore of its separately rounded output variables. The fixed rule updates $N \leftarrow N(50/h)$ with a 22-rpm step limit, holds line speed fixed, and corrects heating outside $[196, 224]$ °C. The separately maintained simulator provides a finite-grid reference $J^* = 89.894$, not a proven global optimum: its grid ties the three zone temperatures and varies temperature from 195 to 225 °C in 2.5 °C steps, screw speed from 50 to 200 rpm in 5-rpm steps, and line speed from 20 to 120 m/min in 2.5-m/min steps, at a fixed 1.0-mm die gap. Unlike the online controller, the reference can vary line speed; the ratio therefore compares different action sets, not two equally equipped optimizers. Energy and throughput use issued setpoints rather than independent sensor measurements.

The archive calls its two-post-write mean J_{end} ; we denote it $\bar{J}$ to avoid confusing it with the final observation J_2 . The mean $\bar{J}/J^*$ across seeds is 0.968 and mean wall time is 36.178 s per seed, including three 12-s waits but excluding reset, initialization, and warmup. Table III exposes both post-write observations: J_2 remains above J_0 but below J_1 in every seed, so improved thickness does not imply a monotonically improving composite score. Final thickness meets the reward band, and the residual gap to the reference lies in the quality, energy, and throughput terms rather than in thickness. Because

(a) Archived thickness observations



(b) Governed-write ledger

Write	Parameter / setpoint change	Transport
1	Casting speed 32.0 → 33.1 m/min	Modbus TCP
2	MDO fast roll 118.0 → 120.8 m/min	Modbus TCP
3	TDO outlet width 3000.0 → 3050.0 mm	OPC UA
4	Casting speed 33.1 → 33.7 m/min	Modbus TCP

4 records; logged keep; parent COMPLETED.

Frozen B; simulated plant; scripted policy. Markers are returned-bucket means, not a continuous trajectory.

Fig. 4. Recorded scripted AgentTeam execution in *B*. (a) The five archived thickness observations against the $25.0 \pm 0.7 \mu\text{m}$ objective; markers are returned-bucket means from queries that requested one-second aggregation, and raw buckets and timestamps are not retained, so no continuous trajectory is inferred. (b) The four governed writes, each named by parameter, setpoint change, and transport. Every write opened an optimization record that received a logged keep, and the parent task reached COMPLETED. The plant is simulated and the policy is scripted, not an LLM-generated search.

the controller inverts the simulator's own $h \propto N/v$ relation, this experiment shows loop closure under a known model, not general search efficiency or learned control superiority.

E. Q3: A Recorded Multi-Node Team Mission

The first mission links task delegation, a five-minute acquisition query, a governed temperature write from 200 to 204.704 °C, a 204.700 °C setpoint-register readback, an attributed keep record, and parent-task completion. All six workflow checks pass; the ± 0.75 °C tolerance makes this a workflow-closure test, not evidence of independent process temperature convergence. The harness issues the industrial-tool calls while mock lead/worker tasks exercise delegation and completion; task completion is not proven to be causally gated by objective attainment.

The BOPET mission extends this workflow to nine simulated devices with 30 writable setpoints and 19 acquisition nodes, against a thickness objective of $25.0 \pm 0.7 \mu\text{m}$. Figure 4 reconstructs the actual intervention sequence from *B* rather than an illustrative optimization curve. Four of six allowed writes act on three distinct parameters: casting speed from 32 to 33.1 m/min, fast-roll speed from 118 to 120.8 m/min, outlet rail width from 3000 to 3050 mm, then casting speed again to approximately 33.7 m/min. The archived thickness observations are 26.90, 26.82, 26.46, 26.00, and 25.48 μm, so absolute target error falls from 1.90 to 0.48 μm and the final observation lies 0.22 μm inside the permitted band. This is attainment at the recorded endpoint, not evidence of sustained convergence or disturbance rejection.

In this mission, each accepted write creates a distinct optimization record, followed by a post-wait gauge observation and a scripted keep judgment. The polling exit rule compares the latest thickness with the pre-write value or exits after its time limit; it does not verify stationarity of successive

observations. The policy uses relative-thickness-error gains of 0.5, 0.35, and 0.3 with fixed actuator rotation, so it does not discover the sequence. The mission logs no independent post-write register value for these actuators and the caller issues keep unconditionally, so the demonstrated contribution is scoped, attributable multi-node execution toward one objective rather than an independently validated judgment or an optimal recipe window. Melt-temperature observations remain between 291.1 and 291.525 °C, inside the configured 268–300 °C window; no disturbance tests that bound.

F. Repeatability and Limits of the Evidence

The immediately preceding same-seed, same-hash archive *A* (20260920093330-md4) also passes 75/75 checks, rejects 24/24 primary and 9/9 second-scenario probes, accepts six controller writes, and closes the BOPET mission with four writes on three parameters. Their final thicknesses nevertheless differ, at 25.64 versus 25.48 μm, as do mean controller $\bar{J}/J^*$ ratios (0.970 versus 0.968) and backstop verdict-observation latencies (130.182 versus 130.205 s). These are two consecutive regression runs, not a reliability campaign. The comparator reports matching verdicts, but its controller counter rows contain null fields and its protocol arrays are empty, so we check the underlying run records rather than treat its "REPRODUCIBLE" label as proof of full numerical agreement.

Earlier archives include process collapse despite a non-failing hard gate; those different-revision runs are not pooled with *A* and *B* or erased by their success. Initial simulator state, sampling timing, and shared-instance residue remain confounders, and the biax journal check has an empty evidence array although the mission log contains a casting-node journal excerpt. These limitations motivate pinned initial conditions,

explicit missing-evidence failures, broader fault injection, and physical validation before deployment.

VI. DISCUSSION AND LIMITATIONS

A. What the Evidence Establishes

The main result is a traceable supervisory path from member authority to multi-node intervention and observed response. Primary archive *B* links protocol-level constraint checks to a scripted team mission that reaches its thickness band. The interlock ablation identifies the mechanism responsible for rejecting the tested window violations, while the frozen-process drill shows why command acceptance and process response need separate checks. Together, these results support an inspectable execution layer across the implemented drivers, rather than relying on task completion as evidence of successful control.

This separation guides diagnosis. A binding or range rejection calls for revising authority or the proposal; a transport or readback failure calls for inspecting execution; an accepted write with an adverse response calls for a new strategy or compensation. A *keep* records the caller's verdict, not independent confirmation of process success. The scripted mission thus demonstrates coordinated execution toward an objective, while the fixed-controller trials isolate loop closure under a known model. Neither measures LLM decision quality or comparative optimization performance.

B. Reproducibility and Threats to Validity

The archives retain commands, seeds, source fingerprints, and per-check evidence, enabling reruns and comparisons. Reconstructing a run also requires the separately versioned simulator and its reset state: initial conditions, asynchronous execution, and sampling windows can change outcomes despite deterministic policies. Harness fingerprints do not pin that simulator, and working-tree hashes can depend on line endings.

The tests and performance reference are internal to the study. Checks were developed against this implementation, and J^* derives from the same simulator used by the controller. Historical warning-only mission outcomes show the limits of aggregate regression gates. The current archive also has a journal check with an empty evidence array (Sec. V-F); retained observation aggregates omit raw buckets and timestamps. Observation-quality gating is not enforced, so stale or unrepresentative telemetry can affect judgments. Optimization records lack a run identifier, and their time-based queries can span run boundaries. Historical real-LLM campaigns used different scoring inputs and observation windows and are not comparable optimization evidence.

External validity remains limited to the simulated scenarios: no physical-device validation, TEP or SWaT replay, human-operator study, or external-framework comparison is reported. Configuration reuse within the implemented drivers does not establish transfer to other equipment or production regimes.

C. Deployment Boundaries

Execution and recovery are not atomic. Bookkeeping may fail after actuation; readback agreement does not establish process recovery, and multi-node compensation can partially succeed. Cross-agent oscillation within permitted ranges, long-horizon contention, storage-failure availability, and worst-case recovery time remain unevaluated. The backstop's sampling and recheck cadence is not a recovery deadline. At its retained-history threshold $K = 2$, escalation replaces automatic restoration and can remain unresolved without an operator.

Bindings govern sanctioned tools, not arbitrary I/O by a privileged or compromised engine. Deployment requires least privilege, endpoint security, and IT/OT separation [25], [37]. Controller interlocks and required IEC 61511 protection must remain independent: governed writes are neither a certified safety function nor a substitute for plant protection [38].

Manual approval introduces risks of fatigue and over-reliance [29], [30], [39], [40]. The present decision endpoint lacks line- or owner-scoped approver authorization and treats an omitted decision flag as approval. Fail-closed decision validation, scoped approver roles, protection of recipe-window edits, and enforced segregation of duties are therefore deployment prerequisites, not demonstrated guarantees. Operational validation must also address approval-channel and alarm-path failures [41], crash-orphaned records, synchronized clocks, and off-host evidence.

VII. CONCLUSION

AGENTWORKSHOP connects agent teams to industrial supervision through member-node capabilities, production context, and intervention records. Its governed loop keeps authorization, device acceptance, submitted judgment, and compensation distinct. Protocol checks, the interlock ablation, and scripted simulated missions establish how these mechanisms support traceable supervisory search. Physical validation, stronger execution consistency, observation-quality gating, and controlled agent comparisons are the next steps.

DATA AVAILABILITY AND DISCLOSURE

The archives identified in Sec. V are retained in the local project, not presented as a publicly released repository. Integrated archives contain `run.json`, `summary.json`, `metrics.csv`, and `agentteam-*.log` traces; the ablation retains `run.json` and `e1-lite.csv`; protocol and static suites retain `run.json` and `reports`. Figure scripts and archive-consistency checks accompany the local source. Reproduction also requires the separately versioned simulator. Figs. 1–3 are conceptual or implementation schematics; Fig. 4 plots archived observations without interpolation. Original console crops remain in the local figure sources, but are not used as experimental measurements. AI assistants supported manuscript editing, evidence checking, and figure preparation under author direction; the authors remain responsible for claims and data.

REFERENCES

- [1] M. Raza, Z. Jahangir, M. B. Riaz, M. J. Saeed, and M. A. Sattar, "Industrial applications of large language models," *Sci. Rep.*, vol. 15, 2025, art. no. 13755.
- [2] S. Colabianchi, F. Costantino, and N. Sabetta, "Assessment of a large language model based digital intelligent assistant in assembly manufacturing," *Comput. Ind.*, vol. 162, 2024, art. no. 104129.
- [3] Z. Liu, R. Zeng, D. Wang, G. Peng, X. Liu, Q. Liu, P. Liu, W. Wang, and J. Wang, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026.
- [4] M. Fakihi, R. Dharmaji, Y. Moghaddas, G. Quiros, O. Ogundare, and M. A. Al Faruque, "LLM4PLC: Harnessing large language models for verifiable programming of PLCs in industrial control systems," in *Proc. 46th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2024, pp. 192–203.
- [5] J. Vyas, M. S. Gill, A. Markaj, F. Gehlhoff, and M. Mercangöz, "From detection to action: Using LLM agents for fault-tolerant control," *arXiv preprint arXiv:2606.28011*, 2026.
- [6] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025.
- [7] P. Leitão, "Agent-based distributed manufacturing control: A state-of-the-art survey," *Eng. Appl. Artif. Intell.*, vol. 22, no. 7, pp. 979–991, 2009.
- [8] J. Zhang, H. Wang, Z. Jia, J. Dong, and L. Ren, "CoMA-IKG: LLM-driven multiagent framework for automated construction of industrial knowledge graph," *IEEE Trans. Ind. Inform.*, vol. 22, no. 6, pp. 5553–5564, 2026.
- [9] P. Liu, L. Qian, X. Zhao, and B. Tao, "Joint knowledge graph and large language model for fault diagnosis and its application in aviation assembly," *IEEE Trans. Ind. Inform.*, vol. 20, no. 6, pp. 8160–8169, 2024.
- [10] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, "LLM experiments with simulation: Large language model multi-agent system for simulation model parametrization in digital twins," in *Proc. IEEE ETFA*, 2024, pp. 1–4, *arXiv preprint arXiv:2405.18092*.
- [11] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, *arXiv preprint arXiv:2505.02076*.
- [12] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. 2025 IEEE 64th Conference on Decision and Control (CDC)*, 2025, pp. 172–179.
- [13] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber-physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025.
- [14] Z. Zhao, D. Tang, C. Liu, L. Wang, Z. Zhang, H. Zhu, K. Chen, Q. Nie, and Y. Ji, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888.
- [15] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [16] LangChain, Inc., "LangGraph," [Online]. Available: <https://github.com/langchain-ai/langgraph>, 2024, accessed: Sep. 20, 2026.
- [17] CrewAI, Inc., "CrewAI," [Online]. Available: <https://github.com/crewAIInc/crewAI>, 2024, accessed: Sep. 20, 2026.
- [18] Anthropic, "Model context protocol," [Online]. Available: <https://modelcontextprotocol.io>, 2024, accessed: Sep. 20, 2026.
- [19] Linux Foundation, "Agent2Agent (A2A) protocol," [Online]. Available: <https://a2a-protocol.org>, 2025, accessed: Sep. 20, 2026.
- [20] International Society of Automation, "ANSI/ISA-95.00.01: Enterprise-control system integration," Standard, 2010.
- [21] —, "ISA-88.00.01-2010—batch control—part 1: Models and terminology," Standard, 2010.
- [22] International Organization for Standardization, "ISO 23247: Digital twin framework for manufacturing," Standard, 2021.
- [23] International Electrotechnical Commission, "IEC 63278-1:2023: Asset administration shell for industrial applications—part 1: Asset administration shell structure," International Standard, 2023.
- [24] A. Gautam, M. R. Aryal, S. Deshpande, S. Padalkar, M. Nikolaenko, M. Tang, and S. Anand, "IIoT-enabled digital twin for legacy and smart factory machines with LLM integration," *J. Manuf. Syst.*, vol. 80, pp. 511–523, 2025.
- [25] International Electrotechnical Commission, "IEC 62443-3-3: System security requirements and security levels," Standard, 2013.
- [26] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, 2018.
- [27] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [28] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001.
- [29] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000.
- [30] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983.
- [31] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *Proc. ICLR*, 2024, *arXiv preprint arXiv:2308.03688*.
- [32] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, " τ -bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.
- [33] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Comput. Chem. Eng.*, vol. 17, no. 3, pp. 245–255, 1993.
- [34] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. 11th Int. Conf. Critical Information Infrastructures Security (CRITIS)*, 2017, pp. 88–99.
- [35] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90.
- [36] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*, USENIX Association, 2024, pp. 1831–1847. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupe>
- [37] National Institute of Standards and Technology, "NIST SP 800-82 rev. 3: Guide to operational technology (OT) security," Special Publication, 2023.
- [38] International Electrotechnical Commission, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector—part 1: Framework, definitions, system, hardware and application programming requirements," International Standard, 2016.
- [39] J. D. Lee and K. A. See, "Trust in automation: Designing for appropriate reliance," *Hum. Factors*, vol. 46, no. 1, pp. 50–80, 2004.
- [40] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010.
- [41] Engineering Equipment and Materials Users Association, "EEMUA 191: Alarm systems—a guide to design, management and procurement," Guide, Edition 3, 2013.